Lab2 实验报告 24300160046 章子墨

一、关卡 1

```
40209d: mov    $0x1,%esi
```

由该行汇编代码可知，关卡 1 的口令是一个整数。

```
4020c2: lea    -0x14(%rbp),%rdx
4020c6: mov    $0x403189,%esi
4020cb: mov    %rbx,%rdi
4020ce: call   4011e0 <_init+0x1e0>
4020d3: cmp    $0x1,%eax
4020d6: jne    4020b3 <phase_01+0x23>
4020d8: mov    %r12,%rdi
4020db: call   402050 <p1_compute>
4020e0: cmp    %eax,-0x14(%rbp)
```

关键汇编，调用_init 函数比较口令是否为一个数，调用 p1_compute 函数得到 challenge 经过计算之后的结果，并与口令进行比较。

接下来在 gdb 中使用 disassemble 命令，得到 p1_compute、p1_step1、p1_step0、p1_step2 这四个函数的汇编：

```
(gdb) disassemble /r p1_compute
Dump of assembler code for function p1_compute:
   0x0000000000402050 <+0>:     f3 0f 1e fa     endbr64
   0x0000000000402054 <+4>:     55      push   %rbp
   0x0000000000402055 <+5>:     48 89 fa        mov    %rdi,%rdx
   0x0000000000402058 <+8>:     48 89 f8        mov    %rdi,%rax
   0x000000000040205b <+11>:    48 c1 e8 20     shr    $0x20,%rax
   0x000000000040205f <+15>:    48 89 d6        mov    %rdx,%rsi
   0x0000000000402062 <+18>:    89 c7   mov    %eax,%edi
   0x0000000000402064 <+20>:    48 c1 ee 10     shr    $0x10,%rsi
   0x0000000000402068 <+24>:    48 89 e5        mov    %rsp,%rbp
   0x000000000040206b <+27>:    e8 c0 ff ff ff  call   0x402030 <p1_step1>
   0x0000000000402070 <+32>:    89 fe   mov    %edi,%esi
   0x0000000000402072 <+34>:    89 d7   mov    %edx,%edi
   0x0000000000402074 <+36>:    89 c1   mov    %eax,%ecx
   0x0000000000402076 <+38>:    e8 a5 ff ff ff  call   0x402020 <p1_step0>
   0x000000000040207b <+43>:    89 ce   mov    %ecx,%esi
   0x000000000040207d <+45>:    89 c7   mov    %eax,%edi
   0x000000000040207f <+47>:    e8 bc ff ff ff  call   0x402040 <p1_step2>
   0x0000000000402084 <+52>:    5d      pop    %rbp
   0x0000000000402085 <+53>:    c3      ret
End of assembler dump.
```

```
(gdb) disassemble /r p1_step1
Dump of assembler code for function p1_step1:
   0x0000000000402030 <+0>:     f3 0f 1e fa     endbr64
   0x0000000000402034 <+4>:     8d 04 bf        lea    (%rdi,%rdi,4),%eax
   0x0000000000402037 <+7>:     01 f0   add    %esi,%eax
   0x0000000000402039 <+9>:     35 63 c2 00 00  xor    $0xc263,%eax
   0x000000000040203e <+14>:    c3      ret
End of assembler dump.
(gdb) disassemble /r p1_step0
Dump of assembler code for function p1_step0:
   0x0000000000402020 <+0>:     f3 0f 1e fa     endbr64
   0x0000000000402024 <+4>:     8d 86 77 c3 00 00       lea    0xc377(%rsi),%eax
   0x000000000040202a <+10>:    31 f8   xor    %edi,%eax
   0x000000000040202c <+12>:    c3      ret
End of assembler dump.
(gdb) disassemble /r p1_step2
Dump of assembler code for function p1_step2:
   0x0000000000402040 <+0>:     f3 0f 1e fa     endbr64
   0x0000000000402044 <+4>:     29 f7   sub    %esi,%edi
   0x0000000000402046 <+6>:     8d 84 7f 29 75 00 00    lea    0x7529(%rdi,%rdi,2),%
eax
   0x000000000040204d <+13>:    c3      ret
End of assembler dump.
```

根据这四个函数的汇编，可以推理出返回的结果是经过这四个函数的调用后的结果，具体的计算关系恢复如下：

```
uint32_t a = (uint32_t)(challenge >> 32);
uint32_t b = (uint32_t)(challenge >> 16);
uint32_t c = (uint32_t)(challenge);

uint32_t r1 = (5*a + b) ^ UINT32_C(0xc263);
uint32_t r0 = (a + UINT32_C(0xc377)) ^ c;
uint32_t r2 = 3 * (r0 - r1) + UINT32_C(0x7529);
```

使用 seed 42 测试通过，结果如下：

```
Phase 1 challenge: 0x28efe333b266f103
Input (unsigned decimal integers):
Phase 1 defused.

[1 已解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

$ python3 public_test.py --phase 1
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Public tests: 10/10
```

使用列表以外的种子（31）进行测试，测试通过：

```
$ ./defuser --seed 31 --phase 1 | ./bomb --seed 31 --phase 1
```

```
BOMBLAB 2026                                        这里还没有明天

                            *
                             \
                          .-"""-.
                        .'       '.
                       /    LOOP   \
                      |             |
                       \           /
                        '---------'

[1 当前] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0xbcd4dd2543bf3fd8
Input (unsigned decimal integers):
Phase 1 defused.

[1 已解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

二、关卡 2

```
4020fd: mov    $0x6,%esi
```

```
(gdb) x/s $rdi
0x7fffffffe2b0: "0 0 0 0 0 0"
(gdb) p/x $rsi
$1 = 0x47526757130f9f52
```

由这两处汇编代码和输出可以得到，本关的口令是六个整数，口令和 challenge 分别在%rdi 和%rsi 中。

```
402128: mov    %eax,%ebx
40212a: lea    -0x2c(%rbp),%rax
40212e: lea    -0x3c(%rbp),%rcx
402132: mov    $0x40317a,%esi
402137: push   %rax
402138: lea    -0x30(%rbp),%rax
40213c: lea    -0x40(%rbp),%rdx
402140: mov    %r12,%rdi
402143: push   %rax
402144: lea    -0x34(%rbp),%r9
402148: lea    -0x38(%rbp),%r8
```

由这一部分的汇编代码可以得知，内存中用来存放口令中六个整数的是一个连续的、总共有 24 个字节的空间（地址从 rbp - 0x40 一直到 rbp - 0x2c）。由于处理输入的函数调用参数时寄

存器一般遵循 rdi,rsi,rdx,rcx,r8,r9，栈这个顺序，故可以推测 rbp - 0x40 处存放了下标为 0 的输入，地址越靠近栈底，下标越大。

```
40215a: mov    %r13,%rax
40215d: mov    -0x40(%rbp),%esi
402160: shr    %rax
402163: and    $0x1f,%eax
402166: lea    (%rax,%rax,2),%eax
402169: cmp    %eax,%esi
```

```
(gdb) p/u $eax
$2 = 27
```

```
(gdb) p/u (($challenge >> 1) & 0x1f)*3
$4 = 27
```

该部分单独将 challenge 的计算值与下标为 0 的输入进行比较，对应的公式应该为((challenge >> 1) & 0x1f)*3，经 gdb 验证，公式正确。

```
40216d: mov    $0x1a,%ecx
402172: mov    $0x1,%edx
402177: mov    %r13,%rax
40217a: lea    (%rsi,%rsi,2),%esi
40217d: shr    %cl,%rax
402180: sub    %edx,%esi
402182: and    $0xf,%eax
402185: add    $0xa,%eax
402188: xor    %esi,%eax
40218a: mov    -0x40(%rbp,%rdx,4),%esi
40218e: cmp    %eax,%esi
402190: jne    402115 <phase_02+0x25>
402192: add    $0x1,%rdx
402196: add    $0x5,%ecx
402199: cmp    $0x6,%rdx
40219d: jne    402177 <phase_02+0x87>
40219f: jmp    402117 <phase_02+0x27>
```

40219d 处的代码说明程序可能会跳转到 402177 处，重新执行代码块，这说明这一部分的代码执行的是循环的功能。rdx 中存放下标，每一次循环过后下标加一，当下标等于 6 时，循环终止；rcx 中存放右移量，一开始是 26，每经过一次循环，右移量增加 5；一开始 rsi 中存放的是上一个下标对应的值用于计算，最后将计算值和本次循环对应的下标做比较。经过这样的分析可以得到下标为 1~5 的口令输入值的计算公式：

```
uint32_t prev = values[0];
uint32_t move = 26;
for (size_t i = 1; i < count; i++){
    uint32_t prev_c = prev*3 - i;
    uint32_t result = ((challenge >> move) & UINT32_C(0xf)) + 10 ;
    values[i] = result ^ prev_c;
    prev = values[i];
    move+=5;
}
```

使用 seed42 和 seed31，均通过测试：

```
Phase 2 challenge: 0x47526757130f9f52
Input (unsigned decimal integers):
Phase 2 defused.

[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

$ python3 public_test.py --phase 2
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Public tests: 10/10
```

```
$ ./defuser --seed 31 --phase 2 | ./bomb --seed 31 --phase 2
──────────────────────────────────────────────────────────────
BOMBLAB 2026                                  这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 待解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

──────────────────────────────────────────────────────────────

单关调试：只记录本次通过的关卡，不播放完整结局。

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x813dde282518d4d5
Input (unsigned decimal integers):
Phase 2 defused.

[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

三、关卡 3

```
4022bd: mov    $0x2,%esi
```

```
(gdb) p/x $rsi
$1 = 0x581ce1ff0e4ae394
(gdb) x/s $rdi
0x7fffffffe2d0: "0 0"
```

```
4022e2: lea    -0x14(%rbp),%rcx
4022e6: lea    -0x18(%rbp),%rdx
```

本关的口令是两个整数，口令和 challenge 分别在%rdi 和%rsi 中。第一个整数保存在%rbp - 0x18 地址处（input[0]），第二个整数保存在%rbp - 0x14 地址处（input[1]）。

```
4022fc: mov    %r12d,%edi
4022ff: xor    %eax,%eax
402301: and    $0x7,%edi
402304: cmp    %edi,-0x18(%rbp)
```

此处可得 input[0]的计算公式：（challenge 后 32 位）^0x7，结果存储在%edi 中。这个变量在后续的 switch 结构中会用到，将其命名为 selector。

```
402309: mov    %r12,%rax
40230c: mov    %r12,%rsi
40230f: shr    $0x3,%rax
402313: shr    $0x20,%rsi
402317: xor    %eax,%esi
```

此处得到新变量 x 的计算公式：(challenge>>32)^(challenge>>3)，该值存放在%esi 中。

```
Dump of assembler code for function p3_dispatch:
   0x0000000000402250 <+0>:     f3 0f 1e fa     endbr64
   0x0000000000402254 <+4>:     89 f8   mov    %edi,%eax
   0x0000000000402256 <+6>:     89 f7   mov    %esi,%edi
   0x0000000000402258 <+8>:     83 f8 07        cmp    $0x7,%eax
   0x000000000040225b <+11>:    77 48   ja     0x4022a5 <p3_dispatch+85>
   0x000000000040225d <+13>:    3e ff 24 c5 20 58 40 00 notrack jmp *0x405820(,%rax,8)
   0x0000000000402265 <+21>:    0f 1f 00        nopl   (%rax)
   0x0000000000402268 <+24>:    e9 63 ff ff ff  jmp    0x4021d0 <p3_case_2>
   0x000000000040226d <+29>:    0f 1f 00        nopl   (%rax)
   0x0000000000402270 <+32>:    e9 4b ff ff ff  jmp    0x4021c0 <p3_case_1>
   0x0000000000402275 <+37>:    0f 1f 00        nopl   (%rax)
   0x0000000000402278 <+40>:    eb 86   jmp    0x402200 <p3_case_4>
   0x000000000040227a <+42>:    66 0f 1f 44 00 00       nopw   0x0(%rax,%rax,1)
   0x0000000000402280 <+48>:    e9 2b ff ff ff  jmp    0x4021b0 <p3_case_0>
   0x0000000000402285 <+53>:    0f 1f 00        nopl   (%rax)
   0x0000000000402288 <+56>:    e9 63 ff ff ff  jmp    0x4021f0 <p3_case_3>
   0x000000000040228d <+61>:    0f 1f 00        nopl   (%rax)
   0x0000000000402290 <+64>:    eb 9e   jmp    0x402230 <p3_case_6>
   0x0000000000402292 <+66>:    66 0f 1f 44 00 00       nopw   0x0(%rax,%rax,1)
   0x0000000000402298 <+72>:    eb a6   jmp    0x402240 <p3_case_7>
   0x000000000040229a <+74>:    66 0f 1f 44 00 00       nopw   0x0(%rax,%rax,1)
   0x00000000004022a0 <+80>:    e9 6b ff ff ff  jmp    0x402210 <p3_case_5>
   0x00000000004022a5 <+85>:    31 c0   xor    %eax,%eax
   0x00000000004022a7 <+87>:    c3      ret
```

程序进入 p3_dispatch 函数，在 gdb 中解析结果如图所示，传入的参数分别是 selector 和 x。

该函数内部是一个分支结构，需要根据 selector 的值判断究竟执行哪个分支。先根据 selector 的值跳转到以 0x405820 开头的、共存储 8 个 8 字节地址的地址表：

```
(gdb) x/8gx 0x405820
0x405820:       0x0000000000402278      0x0000000000402280
0x405830:       0x0000000000402288      0x0000000000402290
0x405840:       0x0000000000402298      0x00000000004022a0
0x405850:       0x0000000000402268      0x0000000000402270
```

```
Dump of assembler code for function p3_case_4:
   0x0000000000402200 <+0>:     f3 0f 1e fa     endbr64
   0x0000000000402204 <+4>:     8d 84 7f 87 9c 00 00    lea    0x9c87(%rdi,%rdi,2),%eax
   0x000000000040220b <+11>:    c3      ret
```

这些地址中存储的数据仍然应该解读为地址，继续跳转到对应的地址，进入对应的 case 函数。比如当 selector 的值为 0 时，先跳转到 0x405820，再跳转到 0x402278，进入 p3_case_4 函数。得到的结果就是最终的 input[1]。

由 0x402258、0x40225b、0x4022a5 这三处汇编代码可得，switch 结构的 default 值应为 0。

使用公开列表中的 seed 和 seed31，均通过测试：

```
$ python3 public_test.py --phase 3
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Public tests: 10/10
```

```
$ ./defuser --seed 31 --phase 3 | ./bomb --seed 31 --phase 3
──────────────────────────────────────────────────────────────

BOMBLAB 2026                                    这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 待解除] [2 待解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

──────────────────────────────────────────────────────────────

单关调试：只记录本次通过的关卡，不播放完整结局。

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0x8a62b6066824196d
Input (unsigned decimal integers):
Phase 3 defused.

[1 待解除] [2 待解除] [3 已解除] [4 待解除] [5 待解除] [6 待解除]
```

四、关卡 4

```
4023ad: mov    $0x1,%esi
```

```
(gdb) p/x $rsi
$1 = 0x9bc585a244823f2
```

```
(gdb) x/s $rdi
0x7fffffffe2b0: "0"
```

本关的口令是两个整数，口令和 challenge 分别在%rsi 和%rdi 中。

```
4023e8: mov    %r12,%rcx
4023eb: mov    %r12,%rax
4023ee: mov    %r12d,%edi
4023f1: xor    %esi,%esi
4023f3: shr    $0x20,%rax
4023f7: shr    $0x7,%rcx
4023fb: and    $0x7f,%edi
4023fe: mov    $0x7f,%edx
402403: xor    %eax,%ecx
402405: call   402330 <p4_walk>
```

```
uint32_t a = ((uint32_t)challenge) & UINT32_C(0x7f);
uint32_t b = 0;
uint32_t c = UINT32_C(0x7f);
uint32_t d = (uint32_t)((challenge >>32) ^ (challenge >> 7));
uint32_t result = phase04_walk(a, b, c, d);
```

计算几个初始值，分别存入变量 a, b, c, d 中，调用 p4_walk 函数。在 p4_walk 函数的汇编代码中可知，这个函数有四个参数。

```
0x0000000000402357 <+39>:    e8 d4 ff ff ff  call   0x402330 <p4_walk>
0x000000000040235c <+44>:    8d 04 40        lea    (%rax,%rax,2),%eax
0x000000000040235f <+47>:    31 d8   xor    %ebx,%eax
0x0000000000402361 <+49>:    48 8b 5d f8     mov    -0x8(%rbp),%rbx
0x0000000000402365 <+53>:    c9      leave
0x0000000000402366 <+54>:    83 f0 11        xor    $0x11,%eax
0x0000000000402369 <+57>:    c3      ret
0x000000000040236a <+58>:    66 0f 1f 44 00 00       nopw   0x0(%rax,%rax,1)
0x0000000000402370 <+64>:    8d 57 ff        lea    -0x1(%rdi),%edx
0x0000000000402373 <+67>:    89 c7   mov    %eax,%edi
0x0000000000402375 <+69>:    e8 b6 ff ff ff  call   0x402330 <p4_walk>
0x000000000040237a <+74>:    8d 04 40        lea    (%rax,%rax,2),%eax
```

```c
uint32_t phase04_walk(uint32_t a, uint32_t b, uint32_t c,uint32_t d){
    uint32_t res = a;
    a = ((c - b)>>1)+b;
    if (res == a){
        return ( a + UINT32_C(0xd7)) ^ d;
    }
    else{
        uint32_t x = a + d ; //ebx
        if ( res < a){
            c = a - 1;
            a = res;
            return ((3 * phase04_walk(a, b, c, d))^x)^UINT32_C(0x1d);
        }
        else{
            b = 1 + a;
            a = res;
            return ((3* phase04_walk(a, b, c, d)) ^ x) ^UINT32_C(0x11);
        }
    }
}
```

根据 p4_walk（报告中仅选取部分）的汇编代码编写辅助函数 phase04_walk 来实现相同的功能。其中地址为 0x402357 和 0x402375 的两行代码执行了递归调用，返回值存储在%rax 中，并继续执行相关的运算操作，直至返回。

```
$ python3 public_test.py --phase 4
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Public tests: 10/10
```

```
$ ./defuser --seed 31 --phase 4 | ./bomb --seed 31 --phase 4
BOMBLAB 2026                                    这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 当前] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x98b9616534c84bb1
Input (unsigned decimal integers):
Phase 4 defused.

[1 待解除] [2 待解除] [3 待解除] [4 已解除] [5 待解除] [6 待解除]
```

使用公开列表中的 seed 和 seed31，均通过测试。

五、关卡 5

```
40248d: mov    $0x2,%esi
```

本关的口令是两个整数，口令和 challenge 分别在%rdi 和%rsi 中。

```
4024cc: lea    -0x14(%rbp),%rsi
4024d0: mov    %r12,%rdi
4024d3: call   402420 <p5_traverse>
```

在调用 p5_traverse 函数之前，%rdi 和%rsi 分别存入 challenge 值和一个地址（视作指针），故可以还原出 p5_traverse 函数的参数类型：uint_64 和 uint_32*

```
0x0000000000402424 <+4>:     48 89 f8        mov    %rdi,%rax
0x0000000000402427 <+7>:     48 89 f9        mov    %rdi,%rcx
0x000000000040242a <+10>:    49 89 f0        mov    %rsi,%r8
0x000000000040242d <+13>:    83 e0 07        and    $0x7,%eax
0x0000000000402430 <+16>:    48 c1 e9 03     shr    $0x3,%rcx
0x0000000000402434 <+20>:    48 8b 14 c5 60 5b 40 00 mov    0x405b60(,%rax,8),%rdx
```

```
(gdb) x/8gx 0x405b60
0x405b60 <p5_heads>:    0x0000000000405be0      0x0000000000405c30
0x405b70 <p5_heads+16>: 0x0000000000405c50      0x0000000000405c10
0x405b80 <p5_heads+32>: 0x0000000000405bd0      0x0000000000405bc0
0x405b90 <p5_heads+48>: 0x0000000000405c40      0x0000000000405bb0
```

进入 p5_traverse 函数内部，得到%eax 内部存储的值是整数 0~7。连续打印从 0x405b60 处开始的 8 个 8 字节数字，得到如图所示的结果。

```
(gdb) p/x $rdx
$2 = 0x405c40
(gdb) x/6wx 0x405c40
0x405c40 <p5_nodes+160>:        0x0000b8c2      0x00000887      0x00405bb0      0x00000000
0x405c50 <p5_nodes+176>:        0x00007b9a      0x00000465
```

以 seed42 的调试为例，%rax 中存储的值为 6，执行完地址为 0x402434 的这行代码之后%rdx 中存储的 0x405c40，像一个地址。再连续打印从 0x405c40 处开始的 6 个 4 字节数字，得到如图所示的结果。

```
0x0000000000402434 <+20>:    48 8b 14 c5 60 5b 40 00 mov    0x405b60(,%rax,8),%rdx
0x000000000040243c <+28>:    48 89 f8        mov    %rdi,%rax
0x000000000040243f <+31>:    48 c1 ef 20     shr    $0x20,%rdi
0x0000000000402443 <+35>:    48 c1 e8 06     shr    $0x6,%rax
0x0000000000402447 <+39>:    31 f8   xor    %edi,%eax
0x0000000000402449 <+41>:    8b 72 04        mov    0x4(%rdx),%esi
```

执行完这些步骤之后，(%rdx)的值是 0xb8c2，(%rdx + 4)的值是 0x887。现在可以推测 0x405b60 是一个链表/数组的起始地址，它的每一个元素都是可以被解读为地址，故它是一个指针数组。每一个指针又对应着一个数组（暂时称为数组 x），暂时无法推测 x 中元素的数据类型。

```
0x0000000000402460 <+64>:    74 1d    je     0x40247f <p5_traverse+95>
0x0000000000402462 <+66>:    b9 01 00 00 00  mov    $0x1,%ecx
0x0000000000402467 <+71>:    48 8b 52 08     mov    0x8(%rdx),%rdx
0x000000000040246b <+75>:    83 c1 01        add    $0x1,%ecx
0x000000000040246e <+78>:    8b 72 04        mov    0x4(%rdx),%esi
0x0000000000402471 <+81>:    03 02    add    (%rdx),%eax
0x0000000000402473 <+83>:    8d 04 80        lea    (%rax,%rax,4),%eax
0x0000000000402476 <+86>:    41 89 30        mov    %esi,(%r8)
0x0000000000402479 <+89>:    31 f0    xor    %esi,%eax
0x000000000040247b <+91>:    39 cf    cmp    %ecx,%edi
0x000000000040247d <+93>:    73 e8    jae    0x402467 <p5_traverse+71>
0x000000000040247f <+95>:    c3       ret
```

这里的汇编代码表示一个循环，执行循环的条件是%ecx 中的值小于等于%edi 中的值（也就是 3）。在每一遍循环中，%rdx 中存储的地址值都增大八字节，同时也要用到 (%rdx) 和 (%rdx + 4）这两个值。可以推测 x 的数据类型是一种结构，它共有两个 uint_32 类型的元素。

```
(gdb) x/4wx 0x405c50
0x405c50 <p5_nodes+176>:        0x00007b9a      0x00000465      0x00405c10      0x00000000
(gdb) x/4wx 0x405c10
0x405c10 <p5_nodes+112>:        0x000030c7      0x00000962      0x00405bd0      0x00000000
```

```
typedef struct {
    uint32_t f0;
    uint32_t f1;
} pair;

static const pair CHAIN[8][4] = {
{{0x00009ddb, 0x00000768}, {0x00007dd2, 0x00000557}, {0x00007b9a, 0x00000465}, {0x000030c7,0x00000962}},
{{0x00007dd2, 0x00000557}, {0x00007b9a, 0x00000465}, {0x000030c7, 0x00000962}, { 0x00006fac, 0x00000a28}},
{{0x00007b9a, 0x00000465}, {0x000030c7, 0x00000962}, {0x00006fac, 0x00000a28}, { 0x000095c8, 0x00000055}},
{{0x000030c7, 0x00000962}, {0x00006fac, 0x00000a28}, {0x000095c8, 0x00000055}, { 0x0000b8c2, 0x00000887}},
{{0x00006fac, 0x00000a28}, {0x000095c8, 0x00000055}, {0x0000b8c2, 0x00000887}, { 0x0000df82, 0x00000106}},
{{0x000095c8, 0x00000055}, {0x0000b8c2, 0x00000887}, {0x0000df82, 0x00000106}, { 0x00003da9, 0x00000251}},
{{0x0000b8c2, 0x00000887}, {0x0000df82, 0x00000106}, {0x00003da9, 0x00000251}, {0x00004b4f, 0x00000614}},
{{0x0000df82, 0x00000106}, {0x00003da9, 0x00000251}, {0x00004b4f, 0x00000614}, { 0x0000acf2, 0x00000b53}}
};
```

在 gdb 中使用“x/4wx + 地址”的命令可以查看每个地址中存储的元素，完善表格。

```
uint32_t phase05_traverse(uint64_t a, uint32_t* out){
    uint32_t selector = (uint32_t)(a & UINT32_C(0X7));
    uint32_t x = (uint32_t)((a >> 3) & UINT32_C(0X3));//rcx,edi
    uint32_t y = ((uint32_t)(a >> 6) ^ (uint32_t)(a >> 32)) ^UINT32_C(0xc89);
    uint32_t val_1 = CHAIN[selector][0].f0;
    uint32_t val_2 = CHAIN[selector][0].f1;
    uint32_t result = ((y + val_1) * 5) ^ (val_2);
    *out = val_2;
    for ( uint32_t i = 1; i <= x; i++){
        val_1 = CHAIN[selector][i].f0;//rdx
        val_2 = CHAIN[selector][i].f1;//rdx+4
        result = ((result + val_1) * 5) ^ val_2;
        *out = val_2;
    }
    return result;
}
```

得到 p5_traverse 函数，最后存入 values 的值分别是 result 和*out。

```
$ python3 public_test.py --phase 5
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Public tests: 10/10
```



使用公开列表中的 seed 和 seed31，均通过测试。

六、关卡 6

```
4025dd: mov    $0x1,%esi
```

```
(gdb) x/s $rdi
0x7fffffffe2d0: "0"
(gdb) p/x $rsi
$1 = 0x37e9671c45376d5d
```

本关的口令是一个整数，口令和 challenge 分别在%rdi 和%rsi 中。

```
0x00000000004025a0 <+32>:    83 e3 38        and    $0x38,%ebx
0x00000000004025a3 <+35>:    83 e0 07        and    $0x7,%eax
0x00000000004025a6 <+38>:    ff 14 c5 00 5e 40 00    call   *0x405e00(,%rax,8)
0x00000000004025ad <+45>:    35 18 eb 00 00  xor    $0xeb18,%eax
0x00000000004025b2 <+50>:    89 c7   mov    %eax,%edi
0x00000000004025b4 <+52>:    ff 93 00 5e 40 00       call   *0x405e00(%rbx)
```

进入 p6_dispatch 函数。在执行地址为 0x4025a6 的代码之前，%eax 中存储的值可能是 0~7 之间的整数，%ebx 中存储的值可能是 0~56 之间 8 的倍数。由此可知，地址为 0x4025a6 的间接调用代码和地址为 0x4025b4 的间接调用代码有相同的功能。

```
(gdb) x/8gx 0x405e00
0x405e00 <p6_ops>:      0x0000000000402550      0x0000000000402560
0x405e10 <p6_ops+16>:   0x00000000004024f0      0x0000000000402510
0x405e20 <p6_ops+32>:   0x0000000000402560      0x0000000000402530
0x405e30 <p6_ops+48>:   0x0000000000402520      0x0000000000402550
```

连续打印从 0x405e00 处开始的 8 个 8 字节数字，这些数字就是由%eax 和%ebx 中的值决定的要调用的函数的地址。

```
(gdb) disassemble /r 0x402550
Dump of assembler code for function p6_op_4:
   0x0000000000402550 <+0>:     f3 0f 1e fa     endbr64
   0x0000000000402554 <+4>:     89 f8   mov    %edi,%eax
   0x0000000000402556 <+6>:     d1 c8   ror    %eax
   0x0000000000402558 <+8>:     c3      ret
End of assembler dump.
```

```
uint32_t phase06_changeValue(uint32_t sele, uint32_t x) {
    switch (sele) {
        case 0: case 7:  // op_4
            return (x >> 1) | (x << 31);
        case 1: case 4:  // op_5
            return ((x >> 7) ^ x) + UINT32_C(0xb1da);
        case 2:  // op_0
            return (x ^ UINT32_C(0x3ded)) + (x << 7);
        case 3:  // op_1
            return (x + UINT32_C(0x51c0)) ^ (x >> 5);
        case 5:  // op_3
            return ((x << 7) + UINT32_C(0x3b9)) ^ (3 * x);
        case 6:  // op_2
            return ((x << 3) | (x >> 29)) ^ UINT32_C(0xdb7);
        default:
            return sele;
    }
}
```

依此解析每个地址，得到函数的表达式。由于每一个函数操作都比较简单，故本题使用 switch

结构实现。

```
$ python3 public_test.py --phase 6
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 10/10
```



使用公开列表中的 seed 和 seed31，均通过测试。

七、Honor

```
(gdb) x/s $rdi
0x7fffffffe3c0: "test"
(gdb) p/x $rsi
$1 = 0xccf635ee9e9e2fa4
```

```
40275c: call   401190 <_init+0x190>//此时rax=0x4
```

```
(gdb) si
0x0000000000402769 in secret_gate ()
1: x/i $pc
=> 0x402769 <secret_gate+73>:   jbe    0x402780 <secret_gate+96>
(gdb) si
0x000000000040276b in secret_gate ()
1: x/i $pc
=> 0x40276b <secret_gate+75>:   add    $0xb0,%rsp
```

先使用“test”作为输入试探炸弹。在执行完地址为 0x40275 的代码（函数调用）之后，%rax 中存储的数字是 0x4（这个调用的功能是计算字符串长度）。由于 0x4<0x24，代码只能跳转到 pop 操作，无法执行 secret_phase 等函数。所以需要对输入进行调整。

```
402786: call   402700 <secret_copy>    /
40278b: cmpl   $0x4e45504f,-0xb0(%rbp)
402795: jne    40276b <secret_gate+0x4b>
```

换用一个更长的输入，在执行完 secret_copy 函数之后无法，和 0x4e45504f 的比较失败了。0x4e45504f 对应 4f、50、45、4e 这四个 ASCII 码，分别代表 O、P、E、N 这四个字母，而%rbp - 0xb0 又对应着字符串的第 32 个字符，故要求输入的字符串满足长度大于等于 36、且从第 32 位开始是“OPEN”这两个条件。将输入更换为“12345678901234567890123456789012OPEN”，重新调试。这次调试可以进入 secret_phase 函数，根据 secret_phase 的汇编代码计算出三个整数口令即可。

```
$ python3 public_test.py --secret
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

```
$ { ./defuser --seed 31; ./defuser --secret --seed 31; } | ./bomb --seed 31
```

```
Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

使用公开列表中的 seed 和 seed31，均通过测试（图太长就不贴了）。

八、辅助工具用途说明

本次实验使用 DeepSeek-V41-Flash 模型进行辅助，主要的辅助方面有：梳理有关汇编语言的知识点、解读实验内容、处理调试过程中碰到的问题等。做了几关之后发现越来越熟练了，在阅读完汇编代码后如果发现操作比较简单就会削减调试的时间，然后让大模型帮助完成机械计算部分的代码。